

State Space Search.

Searching through possibilities to find a good solution.

AI $\Rightarrow$ search + knowledge.

Ex: Travel from kgp to kolkata. Multiple possibilities:
train, bus, car. Compare distance / time / cost.

Search: start $\rightarrow$ goal.

Search examples:

- google search / map.
- chess
- puzzle solving.

We need a precise language to describe search problems.

State space.

Set of all possible situations the system can be in.

Ex: In chess, all possible board configurations.

State space = $\{ \text{all possible states} \}$.

State:

A specific configuration of the system.

Ex: In chess, one particular arrangement of pieces.

State = one element of state space.

State space = all possibilities
State = one possibility

Start State: where we begin (initial configuration)

Ex: In sudoku, the initial puzzle.

Goal State: what we want to reach. (desired configuration)

Ex: In sudoku, a fully solved board

Action: operation allowed in the system.

Ex: In chess, moving a piece.

In sudoku, filling a number.

However, not all actions are allowed in all states. For ex, you can't move a rook diagonally in chess.

Transition: When we apply an action to a state, we get a new state. This is a transition.

Cost: Sometimes actions can have a cost.

Ex: distance in map, number of moves.

Not all problems are just about reaching the goal, sometimes we want the best path.

AI does not solve the problem directly, it searches through the state space.

The most difficult part is the size of the state space.

A classic problem: Water jug problem.

3 water jugs of capacities u, v, w . You can transfer water from one jug to other such that:

- either the source becomes empty.
- or the destination becomes full.

Initial contents: a, b, c

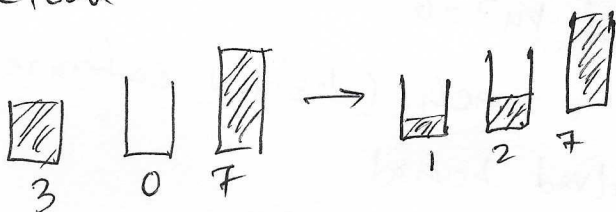
Conservation: $a+b+c = x+y+z$

Desired " : x, y, z

Ex: Jug capacities 3, 5, 7

Start state: 3, 0, 7 } Is it achievable?

Goal " : 1, 2, 7



↓ Yes
State seq.: $(3, 0, 7) \rightarrow (0, 3, 7) \rightarrow (0, 5, 5) \rightarrow (3, 2, 5) \rightarrow (1, 2, 7)$

37

However, valid $\neq$ achievable!

For ex: $(3, 0, 7) \rightarrow (2, 1, 7)$ is not achievable.

Basically the problem is to search whether there is a path from start to goal.

↓

↓
Usually search problems are modeled as graph problems.

Node $\rightarrow$ state, Edge $\rightarrow$ action.

State space as a graph.

HOWEVER the graph can be huge. For ex, think of a bigger instance of the problem with 6 jugs of capacities: 89, 97, 101, 103, 107, 109, & amount of waters = 300.

There are 5,973,255,042 nodes in the graph.

An impossible problem $\rightarrow$ a search space.

Another example: River crossing puzzle.

Goat (G), Wolf (W), Cabbage (C), Farmer (F)

Wolf + goal $\Rightarrow$ not allowed

Groat + Cabbage $\Rightarrow$ "

~~Initial~~ Initial configuration:

Desired

99

1

FGWC

1

FGWC

Is it achievable? If yes, what are the minimum no. of trips required?

1 Aug.

yes, $\downarrow$ min trips = 7 (how?)

We need intelligent searches.

General Search algorithms.

- Uninformed Search: No additional information on the goal node other than the one provided in the problem definition.
Ex: BFS, DFS.
- Informed search: Have information on the goal state, which helps in more efficient searching.
Ex: A* search, Greedy search.

Problem with BFS, DFS: no guarantee of shortest path, for shortest path, Dijkstra can be useful. However we can make a better search algorithm by using heuristic information. This can reduce unnecessary exploration & move towards the goal only along promising paths $\Rightarrow$ A* search.

A* algorithm.

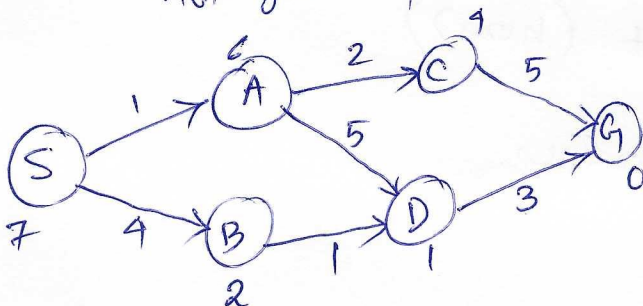
The goal is to find the shortest path from start node to goal node where each node has an estimated, non-negative cost associated with it. This cost is an estimate of the shortest distance from that node to the nearest goal node.

$$\text{A* score of node } v = \underbrace{\text{distance from source to } v}_{(g(n))} + \underbrace{\text{estimated distance from } v \text{ to nearest goal}}_{(h(n))}$$

Visited: Whenever we explore a node, mark it as visited.

Frontier: Assign A* score to the nodes we've encountered but not yet explored. $f(n) = g(n) + h(n)$

Ex



Start state: S
Goal " : G.

(38)

Step-I Start at S. $g(s)=0$ $f(s)=7$.
 $h(s)=7$

Step-II Expand S. Neighbor(s) $\Rightarrow$ A, B.

A: $g(A)=1$ $h(A)=6$ $f(A)=7$
 B: $g(B)=4$ $h(B)=2$ $f(B)=6$

Frontier			
Node	g	h	f
A	1	6	7
B	4	2	6

Choose B

Step-III Expand B. Neighbor(B) $\Rightarrow$ D.

$g(D)=4+1=5$
 $h(D)=1$
 $f(D)=6$

Frontier			
Node	g	h	f
A	1	6	7
D	5	1	6

Choose D.

Step-IV Expand D. Neighbor(D) $\Rightarrow$ G.

$g(G)=5+3=8$
 $h(G)=0$
 $f(G)=8$

Frontier			
Node	g	h	f
A	1	6	7
G	8	0	8

Choose A.

Step-V Expand A. Neighbor(A) $\Rightarrow$ C, D.

C: $g(C)=1+2=3$
 $h(C)=4$
 $f(C)=7$

D: new $g(D)=1+5=6$
 Old $g(D)=5$ (better)

So do not consider D.

Frontier			
Node	g	h	f
C	3	4	7
G	8	0	8

Choose C

Step-IV Expand C. Neighbor (C) $\Rightarrow G$

$g(G) = 3 + 5 = 8$
Same as before,
no improvement.

Frontier			
Node	g	h	f
G	8	0	8

Step-VII: Expand G $\Rightarrow$ goal reached.

Final path: $S \rightarrow B \rightarrow D \rightarrow G$.